

Sitzung 2: rse_annotations – Forschungscode markieren und prüfen

Ein Werkzeug für Responsible RSE

Dr. Julian Dehne

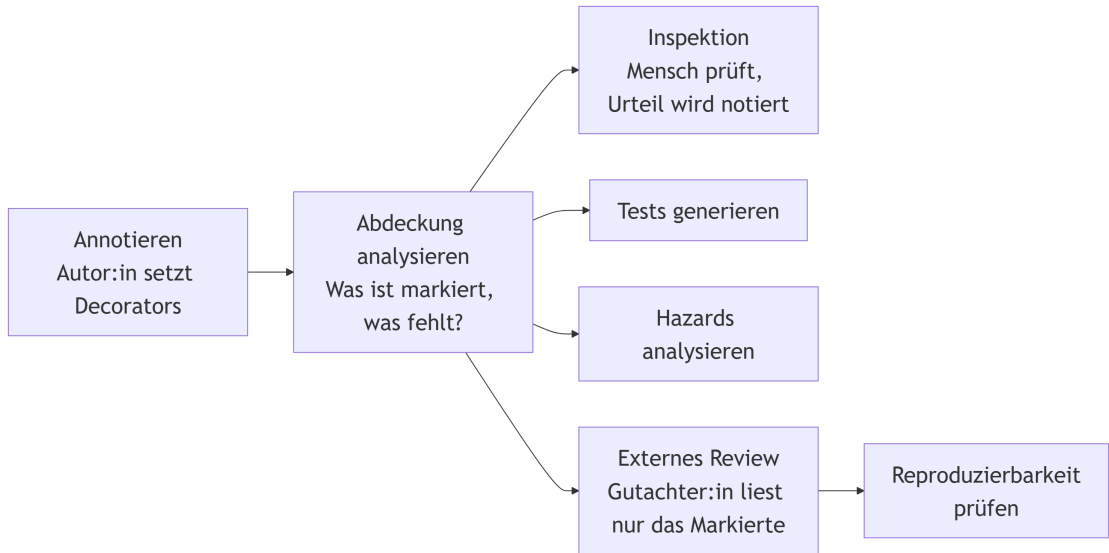
Gesellschaft für Informatik e.V. (GI) · deRSE
julian.dehne@gi.de · ORCID 0000-0001-9265-9619

Idee

- Forschungscode wird zunehmend generiert. Niemand liest tausende Zeilen plausibel aussehenden Codes vollständig.
- **Idee:** Wer den Code schreibt, *markiert* die Stellen, an denen das wissenschaftliche Ergebnis falsch werden kann.
- Die Markierung ist ein **Filter**: Sie verkleinert die Menge an Code, die ein Mensch lesen muss – für die Autorin (eigene Inspektion) wie für die Gutachterin (externes Review).
- Werkzeuge (**Plugins**) führen den Menschen gezielt zu den markierten Stellen.

Die Maschine schlägt vor, der Mensch entscheidet.

Der Prozess



Zwei Unterscheidungen, vier Felder

Woher kommt ein Hazard? (Zeilen) – **Muss der Code dafür laufen?** (Spalten)

	ohne Ausführung (statisch)	zur Laufzeit
erkannt das Werkzeug findet ihn	Hazard-Scan im Quelltext (AST): <code>stochastic, model_call, config;</code> Abdeckung und Kandidaten	Checks mit Fixtures: Ist <code>@functional</code> wirklich rein? Liest <code>@data_input</code> wirklich?
deklariert die Autorin setzt ihn	externes Review der markierten Stellen, z.B. <code>hardware_dependency</code>	Inspektion mit abgeleiteter Formel, generierte Tests

- *Statisch* liest nur, importiert nichts: läuft überall – auch ohne TensorFlow, ohne GPU.
- *Zur Laufzeit* importiert den Code und ruft ihn auf: braucht die Abhängigkeiten und die Hardware.

Ordner	Aufgabe
decorators/	die Markierungen selbst
scan/	statische Analyse, importiert nichts
inspection/	menschliche Prüfung, Urteile
core/	Plugin-Vertrag, Zielprojekt, Befunde
plugins/hazards/	ein Ordner pro Responsible-RSE-Frage

Decorators

Was ist ein Decorator?

```
@functional  
def normalize(v, lo, hi):  
    return (v - lo) / (hi - lo)
```

- Bedeutet: `normalize = functional(normalize)`.
- Läuft **einmal, beim Import** – nicht bei jedem Aufruf.
- Beim Import: Eintrag ins Register. Beim Aufruf: nichts Zusätzliches.

Definitionszeit vs. Aufrufzeit

- In Python führt ein Import Code aus (anders als in Java).
- **Java-Annotationen** sind reine Metadaten, geprüft vom Compiler.
- **Python-Decorators** sind ausführbare Funktionen.

Ein neuer Decorator = **eine Funktion**, markiert mit `@_concern`. Der Hilfetext ist der Docstring.

Jeder Decorator markiert eine Stelle, an der das Ergebnis falsch werden kann, und sagt, worauf man prüfen soll:

- `@functional`: Mathematik
- `@mapping`: Umformung von Daten
- `@data_input` / `@data_output`: Datenein- und -ausgang
- `@hardware_dependency`: Ergebnis hängt von der Hardware ab (z.B. GPU) – Reproduzierbarkeit
- ... die Menge ist offen und wächst.

Einordnung: Decorator-Muster (GoF) und Aspekte (AOP)

	Decorator-Muster (GoF)	Aspekt (AOP)
Quelle	Gamma et al. 1994	Kiczales et al. 1997
Idee	ein Objekt umhüllt ein anderes, gleiche Schnittstelle	ein Querschnittsbelang, einmal formuliert, an vielen Stellen eingewoben
Wo?	wo man das Objekt einpackt	an <i>Join Points</i> , gewählt per <i>Pointcut</i>
Was?	Verhalten vor/nach dem Aufruf	der <i>Advice</i> läuft an jedem Join Point

Bei uns: `@functional` & Co. umhüllen nichts – sie **markieren** nur und geben die Funktion unverändert zurück.

- *Concern* = was zu prüfen ist; *Join Point* = die markierte Funktion.
- *Aspekt* = das Plugin; sein *Advice* ist die Prüfung – sie läuft **im Werkzeug**, nicht beim Aufruf des Forschungscode.

Inspection

- Quelltext und abgeleitete **Formel** zeigen → ja / nein / später.
- Eine Zeile Mathematik gegen die eigene Absicht prüfen. Gerendert, nicht bewiesen.
- `inspection.yaml` hält nur **menschliche Urteile**: “jemand hat das geprüft” wird nachvollziehbar.
- **Kein LLM prüft LLM-Code.** Bewusste Grenze des Werkzeugs.

Maschinelle Prüfung $\neq$ menschliches Urteil

- Automatische Checks liefern **Befunde**: Ist `@functional` wirklich rein? Liest `@data_input` wirklich?
- Nur das Urteil eines Menschen wird als **Verdict** festgehalten.
- Beides getrennt halten – auch im Code.

Plugins

Ein Plugin = eine Frage

- Lizenz, Dual Use, Footprint, ... – jede Responsible-RSE-Frage ist ein Plugin.
- Dieselben Modi für alle: **Inspektion, Analyse, Testgenerierung, externes Review.**
- Neue Frage = neues Plugin, kein Umbau des Frameworks.
- Ein **Stub** ist ein Plugin ohne Implementierung; sein Docstring ist die Aufgabe für Studierende.

Scan

Wie der Scan arbeitet

1. Alle `*.py`-Dateien finden (Tests, Caches, Vendored überspringen).
2. Jede Datei mit `ast.parse` in einen Baum verwandeln – nichts wird importiert oder ausgeführt.
3. Import-Aliase auflösen (`@f aus import functional as f` zählt als `@functional`).
4. Jede Funktion besuchen: Zählt sie mit? Welcher Decorator? Falls keiner: Vorschlag. Welche Hazards?
5. Hazards den Aufrufgraph hinauf vererben (wer eine LLM-Funktion aufruft, erbt `model_call`).
6. Ergebnis: `CoverageReport` – Abdeckung + Kandidatenliste.

Einen Hazard von Hand bauen

Ausgangslage: Code, der eine GPU voraussetzt

examples/gpu_tensors.py

```
import tensorflow as tf
```

```
def add_on_gpu():
```

```
    # kein stilles Ausweichen auf die CPU
```

```
    tf.config.set_soft_device_placement(False)
```

```
    with tf.device("/GPU:0"):
```

```
        a = tf.constant([1.0, 2.0])
```

```
        b = tf.constant([3.0, 4.0])
```

```
    return tf.add(a, b)
```

- Ohne GPU scheitert der Aufruf – **laut**, statt still auf die CPU auszuweichen.
- Dem AST sieht man die Abhängigkeit nicht zuverlässig an → ein **deklarerter** Hazard.
- Wer das Ergebnis reproduzieren will, muss genau diese Stelle lesen – die Markierung führt dorthin.

Schritt 1: Den Decorator definieren

rse_annotations/decorators/markers.py

```
@_concern
def hardware_dependency(fn=None, *, fields=None):
    """Marks code whose result depends on the hardware it runs on
    (e.g. a GPU).

    Review for reproducibility: does the result survive other
    hardware, or does a different device or a CPU fallback change it
    (precision, non-deterministic kernels)?"""
    return _mark(hardware_dependency, fn, fields)
```

Der Docstring ist keine Hardware-Dokumentation, sondern der **Prüfauftrag** an die Gutachterin. Alles Weitere (Enum, Export, Scanner) ergibt sich aus @_concern.

Schritt 2: Den Forschungscode markieren

examples/gpu_tensors.py

```
import tensorflow as tf

from rse_annotations.decorators.markers import hardware_dependency

@hardware_dependency
def add_on_gpu():
    tf.config.set_soft_device_placement(False)
    with tf.device("/GPU:0"):
        ...
```

Zwei Zeilen: ein Import, ein Decorator. Der Rumpf bleibt unverändert.

Schritt 3: Prüfen, ob die Markierung ankommt

```
python -m rse_annotations examples --coverage
```

By review concern

concern	decorated	candidates
@functional	3	0
@mapping	2	1
@data_input	1	0
@data_output	1	1
@hardware_dependency	1	0

By file

file	funcs	decorated	coverage	candidates
gpu_tensors.py	1	1	100%	0
sample_pipeline.py	7	7	100%	0

Vorher stand `gpu_tensors.py` bei 0%. TensorFlow muss dafür **nicht** installiert sein.

Schritt 4: Das Werkzeug starten, Option 4 wählen

```
python -m rse_annotations examples
```

```
Found 8 decorated function(s): 3 @functional, 2 @mapping, 1 @data_input,  
1 @data_output, 1 @hardware_dependency
```

Choose a mode:

```
1) Human inspection  -- a person reviews flagged code; verdicts are recorded  
2) Hazard analysis   -- every plugin examines the code and reports findings  
3) Test generation   -- write test scaffolds  
4) External review   -- an outside reviewer inspects the marked code; writes a protocol  
> 4
```

External review of examples

```
8 marked snippet(s); nothing is imported or run.  
Reviewer (name or role) > JOSS-Gutachterin
```

Ohne Menü: `python -m rse_annotations examples --review`

Schritt 5: Als Gutachterin jede markierte Stelle beurteilen

`== @hardware_dependency -- Review for reproducibility: does the result survive other hardware, or does a different device or a CPU fallback change it (precision, non-deterministic kernels)?`

```
[8/8] add_on_gpu      (gpu_tensors.py:7)
--- source ---
@hardware_dependency
def add_on_gpu():
    tf.config.set_soft_device_placement(False)    # no silent fallback to the CPU
    with tf.device("/GPU:0"):
        a = tf.constant([1.0, 2.0])
        b = tf.constant([3.0, 4.0])
        return tf.add(a, b)
```

Does this snippet hold up for @hardware_dependency? [y]es / [n]o / [s]kip > n

Note for the protocol (Enter for none) > nur mit GPU reproduzierbar; kein CPU-Fallback

Die Gutachterin liest **8 markierte Stellen** statt des ganzen Repositorys – und erfährt je Stelle, worauf es für die Reproduzierbarkeit ankommt.

Schritt 6: Das Protokoll lesen

Recorded 8 finding(s): 5 accepted, 2 declined, 1 pending.

Scope: 8 of 10 eligible function(s) marked (80%); unmarked code was not reviewed.

Protocol written to examples/review_protocol.yaml

```
target: "examples"
reviewer: "JOSS-Gutachterin"
date: "2026-10-01"
scope: "8 of 10 eligible function(s) marked (80%); unmarked code was not reviewed"
findings:
  - function: "add_on_gpu"
    location: "gpu_tensors.py:7"
    concern: "hardware_dependency"
    verdict: "declined"
    note: "nur mit GPU reproduzierbar; kein CPU-Fallback"
```

`scope` nennt die Grenze des Filters: Was nicht markiert ist, wurde nicht begutachtet. Eigene Datei, getrennt von `inspection.yaml` der Autor:innen.

Schritt 7: Die Tests laufen lassen

```
python -m pytest tests/test_external_review.py -v
```

```
test_items_cover_every_concern_in_listing_order PASSED
test_reviewer_is_asked_the_question_of_each_concern PASSED
test_review_writes_a_protocol PASSED
test_rerun_prefills_the_previous_answers PASSED
test_review_shows_the_authors_verdict PASSED
test_cli_review_runs_without_importing PASSED
test_cli_menu_offers_the_review_as_fourth_mode PASSED
```

```
7 passed in 0.16s
```

- Der vorletzte Test ist der wichtige: Das Zielmodul importiert ein Paket, das nicht installiert ist – das Review läuft trotzdem.
- Ganze Suite: `python -m pytest`